

OPERATOR DOCUMENTATION

Platform Administrator Manual

Deploying and maintaining the attendance platform

Platform administrator

Version 2.0 | August 2026 | Firmware v1.8.0

Contents

1. Introduction
2. System architecture
3. Initial deployment
4. Firmware
5. Onboarding an institution
6. Secrets
7. OTA releases
8. Database
9. Troubleshooting
10. Pre-handover checklist

1. Introduction

This manual is for the platform administrator — the developer or operator who deploys and maintains the system. As platform admin you have full access to every institution. Your responsibilities are:

- Deploying and updating the cloud backend (Supabase, edge functions, cron)
- Building and flashing firmware to ESP32 devices
- Onboarding new institutions and creating their first super admin
- Managing secrets, TLS certificates, and OTA releases
- Monitoring the system and diagnosing failures

`platform_admin` is the only role with cross-institution access. Protect these credentials carefully.

1.1 Roles at a glance

Role	Scope	Created by
<code>platform_admin</code>	Every institution	Manually, in SQL
<code>super_admin</code>	One institution, full access	Platform admin, at onboarding
<code>admin</code>	One institution, day-to-day operations	Super admin
<code>teacher</code> / <code>staff</code>	One unit, read-only	Super admin or admin
<code>cashier</code>	One shop: clients, sales, catalog	Super admin or admin

1.2 Institution types

Type	Adds	Removes
<code>school</code>	Academic terms, year-end promotion	—
<code>office</code>	Periods and holidays	Promotion
<code>shop</code>	Clients, sales, catalog, loyalty, reports, cashier role	Promotion, academic terms

2. System architecture

Three tiers, all yours to maintain:

- **ESP32 firmware** — compiled in Arduino IDE, flashed over USB, updated over the air thereafter
- **Supabase** — PostgreSQL, Deno edge functions, `pg_cron`, `pg_net`
- **Next.js dashboard** — deployed on Vercel, talks to Supabase server-side only

All traffic is HTTPS. The ESP32 validates server certificates against a bundled CA chain. The dashboard never exposes Supabase keys to the browser — every query runs server-side.

2.1 Device authentication

Each device holds its **own** secret, issued at provisioning time and stored in `devices.device_secret`. A compromised device exposes only itself.

`devices.revoked` is the server-side kill switch. Setting it true makes the edge functions reject that device immediately, independent of whether the physical unit ever gets wiped.

NOTE

Older builds shared one `institutions.device_secret` across an institution. That column is retained only as a transitional fallback. Once every device in an institution has re-provisioned, the shared path is dead weight — see `docs/device-secret-migration-rollout.md`.

3. Initial deployment

3.1 Supabase

1. Create a Supabase project.
2. Run every migration in `backend/supabase/migrations/`, in filename order.
3. Enable the `pg_net` extension. The mark-absent cron cannot call `net.http_post()` without it.
4. Enable the `pg_cron` extension.
5. Deploy the edge functions: `supabase functions deploy --project-ref <ref>` from `backend/supabase`.
6. Set the `BOOTSTRAP_SECRET` environment variable for the functions. It must match `secrets.h` in the firmware.
7. Add a `CRON_SECRET` vault secret — a strong random string.
8. Schedule the nightly absence job in the SQL editor:

```
select cron.schedule('mark-absent-daily', '0 22 * * *', $$select net.http_post(url :=
'https://<ref>.supabase.co/functions/v1/mark-absent', headers := '{"x-cron-secret":
"<CRON_SECRET>"}'::jsonb)$$);
```

Pick a cron time that falls after midnight in the latest timezone you serve.

IMPORTANT

Migrations carry a "do not apply to cloud blind" header for a reason. Read each header before applying — several are widenings that must land before the matching frontend deploy, and a few are ordering-sensitive.

3.2 Vercel

1. Link a Vercel project to the repository.
2. Set the environment variables: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `NEXTAUTH_SECRET`.
3. Deploy.
4. Create your own auth user in Supabase, then give it the platform role:

```
insert into profiles (id, role) values ('<your-auth-user-uuid>', 'platform_admin');
```

4. Firmware

4.1 Prerequisites

- Arduino IDE 2.x with the Espressif ESP32 board package
- Libraries: Adafruit Fingerprint Sensor, RTCLib, ArduinoJson v7, Adafruit SSD1306, ESP32 OTA Update
- USB cable

4.2 secrets.h

Create `firmware/ClassAttendance_Current_RTC/secrets.h` from `secrets.example.h`. It is gitignored. It defines exactly one thing:

```
#define BOOTSTRAP_SECRET "your_bootstrap_secret_here"
```

This must match the `BOOTSTRAP_SECRET` set on your Supabase edge functions.

There is no compile-time WiFi credential. A device gets WiFi at runtime through its own captive portal: on a factory-fresh boot it broadcasts `Attendance-Setup` (password `setup1234`) with a page at `192.168.4.1`, and whoever is on-site enters the real network details. You never need a client's WiFi password to build or flash their device.

The portal only opens by itself on that first boot. Afterwards the only in-field way back in is a pre-enrolled master fingerprint.

IMPORTANT

Enrol a master fingerprint on every device before it leaves your hands, and make sure the client enrolls one of their own. Without it, a WiFi change means the device must come back to you for a physical reflash. This is the single most common avoidable support call.

4.3 certs.h

`certs.h` holds `ROOT_CA_BUNDLE`, the PEM bundle mbedTLS validates against. It must contain:

- WE1 (Google Trust Services) intermediate — Supabase
- GlobalSign ECC Root CA R4 — Supabase
- Sectigo Public Server Auth CA DV E36 — GitHub API
- Sectigo Public Server Auth Root E46 — GitHub API
- ISRG Root X1 — GitHub CDN, for OTA downloads

mbedTLS does not do AIA fetching. If a server omits an intermediate from its handshake and it is not in this bundle, the connection fails with a confusing "connection refused". Always include intermediates, not just roots.

To capture a live chain when refreshing:

```
$req = [Net.HttpWebRequest]::Create("https://<ref>.supabase.co"); $req.GetResponse() | Out-Null; $req.ServicePoint.Certificate
```

4.4 Flashing

1. Open `ClassAttendance_Current_RTC.ino`.
2. Board: ESP32 Dev Module. Select the COM port.
3. Upload.
4. Watch Serial Monitor at 115200 baud to confirm boot.

Current version is **v1.8.0**. Bump `FIRMWARE_VERSION` on every release.

5. Onboarding an institution

1. Log in as platform admin and go to **Create institution**.
2. Enter the name, pick the **type** — School, Office, or Shop / Retail — and set the timezone.
3. For school and office, choose which member types are tracked and their scan modes. Shops track staff.
4. Enter the first super admin's name, email, and a temporary password.
5. Press **Create**.
6. Hand over the dashboard URL, the credentials, and the right manuals (section 5.2).

Everything else — labels, currency, branding, modules — the super admin configures themselves under **Settings**.

5.1 Creating a super admin by hand

If you prefer SQL: create the user in Supabase Auth, copy the UUID, then

```
insert into profiles (id, role, institution_id) values ('<uuid>', 'super_admin', '<institution-uuid>');
```

5.2 What to hand each client

Institution type	Give them
School	<code>School_Manual.pdf</code> + <code>Super_Admin_Manual.pdf</code> + <code>Admin_Manual.pdf</code> + <code>Staff_Manual.pdf</code>
Office	<code>Office_Manual.pdf</code> + <code>Super_Admin_Manual.pdf</code> + <code>Admin_Manual.pdf</code> + <code>Staff_Manual.pdf</code>
Shop	<code>Shop_Manual.pdf</code> + <code>Super_Admin_Manual.pdf</code> + <code>Admin_Manual.pdf</code>

The type manual explains *what the system does for their kind of business*. The role manuals explain *what each login can do*. Clients need both.

5.3 Institution lifecycle

`institutions.status` controls access:

Status	Effect
<code>active</code>	Normal operation

Status	Effect
suspended	Temporary — e.g. non-payment. Users are bounced to <code>/suspended</code> . Easily reversed.
deactivated	Long-term offboard. Data retained for export.

Both non-active states behave identically at the gate; the difference is operational intent. Enforcement is application-side in `verifySession()`, not RLS, because the dashboard reads via the service role.

Deactivating does **not** wipe devices — they resume on reactivation. Deleting an institution does cascade and does enqueue device wipes.

6. Secrets

6.1 Per-device secrets

Each device's secret lives in `devices.device_secret`, minted by `assignment-poll` when the device is first assigned, and stored on the device in SPIFFS.

To rotate one device: set `devices.revoked = true`, enqueue a reset so it wipes its SPIFFS identity, and let it re-register. It receives a fresh secret when reassigned.

To decommission a device, prefer setting `revoked = true` over a hard delete. Revocation takes effect server-side immediately; the SPIFFS wipe is cooperative and only lands when the device next polls.

6.2 BOOTSTRAP_SECRET

Compiled into firmware. To rotate: update `secrets.h`, update the edge function environment variable, then reflash or push an OTA release. Both sides must change together.

6.3 CRON_SECRET

Update the vault secret, then drop and recreate the cron job so the new value is in the scheduled statement.

7. OTA releases

The firmware checks the GitHub releases API on every boot and updates itself if a newer version exists.

1. In Arduino IDE: **Sketch** → **Export Compiled Binary**.
2. Create a GitHub release tagged `firmware-v<version>`, higher than the current `FIRMWARE_VERSION`.
3. Upload the binary as a release asset named exactly `ClassAttendance_Current_RTC.ino.bin`.
4. Devices pick it up on their next boot.

There is no forced push — devices only check at boot, and update one at a time. The screen shows "OTA UPDATE / Do not unplug" and the sensor breathes red throughout.

8. Database

8.1 Migrations

All schema changes live in `backend/supabase/migrations/`. Apply new ones in the SQL editor in filename order. Never re-run old migrations except when building a fresh database.

8.2 Backups

Supabase runs automated daily backups on Pro and above. For extra safety, take on-demand `pg_dump` exports via the connection string. Point-in-time restore is available from the Supabase dashboard.

8.3 Raw data

The table and SQL editors give the service role full access to everything. There is no undo for destructive SQL.

9. Troubleshooting

Symptom	Likely cause	Fix
"Bootstrap HTTP client error: connection refused"	TLS chain failure	Confirm <code>certs.h</code> includes all intermediates. Test with <code>setInsecure()</code> to prove reachability.
404 "Device not found" on provisioning poll	No <code>devices</code> row for that MAC, or missing <code>provisioning_token</code>	Check the table; confirm the provisioning-token migration ran.
mark-absent: "schema net does not exist"	<code>pg_net</code> not enabled	Enable <code>pg_net</code> in Database → Extensions.
Dashboard "unauthorized" after login	No <code>profiles</code> row, or wrong role	Insert or correct the profile for that auth UUID.
User bounced to /suspended	<code>institutions.status</code> is not <code>active</code>	Set it back to <code>active</code> if this was not intended.
Device re-registers after every SPIFFS clear	Clear ran after <code>loadProvisioning()</code> already cached the id	The clear must be the first thing in <code>setup()</code> , followed by <code>ESP.restart()</code> .
OTA fails fetching GitHub releases	ISRG Root X1 or Sectigo cert missing	Ensure all five certificates are in <code>ROOT_CA_BUNDLE</code> .
Device stuck breathing yellow	Never assigned	Assign it under Devices .
Scans logged but rejected with 401	Device revoked	Check <code>devices.revoked</code> .
Offline scans stuck showing absent	Late-arriving scans after mark-absent ran	Fixed in current builds; confirm the deployed edge functions are current.

10. Pre-handover checklist

Run through this before leaving a client site.

- ☐ Institution created with the correct **type** and **timezone**
- ☐ Super admin can log in and has changed the temporary password
- ☐ Settings configured: labels, currency, scan modes, days tracked, punctuality thresholds, time display, branding
- ☐ Holidays or closed days entered for the current period
- ☐ Every device assigned to a group and unit, showing solid blue
- ☐ **Master fingerprint enrolled on every device** and the client knows whose finger it is

- ☐ Client has successfully enrolled at least one person themselves, unaided
- ☐ Client has recorded a real scan and found it on the dashboard
- ☐ For shops: catalog loaded, one test sale recorded, cashier login working
- ☐ Manuals handed over per section 5.2
- ☐ Client knows how to reach you